

Private Encrypted Data Alignment by ID

A design note: extending OKVS/RLWE Reverse-PMT to payload transport

Working notes

June 10, 2026

Abstract

We sketch how to repurpose the OKVS-based, RLWE-instantiated Reverse-PMT (RPMT) of the companion paper into a *private encrypted data alignment* primitive: a receiver R holds a table $\{(\text{ID}_i, \text{payload}_i)\}$ and a sender S holds a set of IDs; for each of S 's IDs that matches one of R 's, S obtains an *encryption* of the corresponding payload, aligned to S 's own order, so that S can run a homomorphic analysis on the aligned payloads together with its own data. The two non-trivial pieces are (i) neutralizing the OKVS “garbage” on non-matching IDs without leaking the join pattern, via a secret-shared membership indicator and a single ciphertext $\times$ ciphertext mask, and (ii) carrying real-valued payloads in CKKS, which turns the OKVS encoding problem from a finite-field full-rank question into a *numerical conditioning* question. We record the design decisions and the resulting research questions.

1 Setting and target functionality

Let R (receiver, data owner) hold a key–value table $T = \{(\text{ID}_i, \text{payload}_i)\}_{i \in [n_x]}$ and let S (sender) hold a set of identifiers $\{\text{ID}_j^S\}_{j \in [n_y]}$. We want a protocol that delivers to S , for each j , an *encryption* of payload_i whenever $\text{ID}_j^S = \text{ID}_i$ for some i , and an encryption of (effectively) zero otherwise; the result is aligned to S 's own indexing. S then evaluates an application-specific homomorphic function on these aligned ciphertexts and its own plaintext data. Throughout, R holds the RLWE secret key(s); all ciphertexts handed to S are under R 's public key, so S computes but cannot decrypt. The final analysis output is returned to R (or kept secret-shared); this delivery and the choice of analysis are treated as out of scope here.

2 Recap of the base RPMT mechanism

The base construction (companion paper) realizes $F_{\text{RPMT}}: b_j = \mathbf{1}(y_j \in X)$. Its mechanics that we reuse:

1. R OKVS-encodes $\{(x'_i, \text{IS})\}$ — *same* indicator value IS for every key — into p , using the *Random Spaced Band* (RSB) matrix (a fixed column permutation π_N of a random-band matrix, aligning bands to the RLWE slot size N).
2. R RLWE-encrypts p (block-packed, plus pre-rotated copies) and sends it.
3. S homomorphically evaluates the OKVS decoding $M \cdot p$ as a (block-)diagonal matrix–vector product, using only slot-wise PlainMult/CtxtAdd. *Sequencing* (Algorithm 3 of the paper) splits the collision-laden matrix into L layers $\bar{M}_t \in \mathbb{Z}_p^{N \times m}$, each with $\leq w + \zeta$ diagonals; the assignment

of item i to layer/slot (μ_i, τ_i) depends only on the band start $a_i = H_1(k_i)$, the slot count N , and the span limit ζ — *not* on w .

4. Each decoded slot is **IS** if $y_j \in X$ and a pseudorandom value otherwise. The base protocol re-randomizes, returns the L ciphertexts to R, who tests equality with **IS**.

A key efficiency point: sequencing is an n -balls-into- N -bins process, so its batching efficiency relies on the regime $N \ll n$ (well-filled slots). The base parameters fix $N = 2^{13}$ for all $n \in [2^{16}, 2^{22}]$.

Secret-shared PMT (Appendix D). Masking the decoded plaintext by r_μ before R decrypts, followed by a secret-shared equality test $F_{\text{ss-peqt}}$, yields XOR shares $b_{X,i}$ (held by R) and $b_{Y,i}$ (held by S) with $b_{X,i} \oplus b_{Y,i} = \mathbf{1}(y_i \in X)$. We reuse this verbatim for the indicator.

3 The alignment construction

3.1 From indicator to payload: two OKVS instances

Replace the single all-IS OKVS by *two* OKVS instances sharing the same keys $k_i = \text{OPRF}(\text{ID}_i)$:

- an *indicator* OKVS over a finite field (values all = **IS**, or all = 1), used exactly as in the base protocol to produce the membership shares $b_{X,i}, b_{Y,i}$;
- a *payload* OKVS whose value for key k_i is payload_i , decoded homomorphically so that S holds an RLWE ciphertext whose slot for y_j contains payload_i (match) or pseudorandom garbage (non-match).

With a generic payload, the decoded value alone no longer certifies membership (garbage is indistinguishable from a real payload), which is exactly why the separate indicator is needed.

3.2 Removing the garbage without leaking the join

After the indicator ssPMT, S holds the garbage-laden payload ciphertext, its own share $b_{Y,i}$, and knows the slot of each y_j ; R holds $b_{X,i}$. Let $b_i = b_{X,i} \oplus b_{Y,i}$ be the (secret) membership bit. To zero out non-matches while keeping b_i hidden from both parties:

1. R encrypts its share $b_{X,i}$ (packed into the payload slot layout) and sends $\text{Enc}(b_{X,i})$ to S.
2. S forms $\text{Enc}(b_i)$ *using only plaintext operations*, because its own share $s := b_{Y,i}$ is a known constant per slot:

$$b_i = b_{X,i} \oplus s = (1 - 2s) b_{X,i} + s \implies \text{Enc}(b_i) = \text{PlainAdd}(\text{PlainMult}(\text{Enc}(b_{X,i}), 1 - 2s), s). \quad (1)$$

(Note $a \oplus b = a + b - 2ab$, not $a + b - ab$; the latter is OR.) No ciphertext $\times$ ciphertext product is needed here.

3. S multiplies the membership ciphertext by the payload ciphertext:

$$\widetilde{\text{Enc}(\text{payload})} = \text{Enc}(b_i) \odot \text{Enc}(\text{payload-or-garbage}).$$

This is the *single* ciphertext $\times$ ciphertext multiplication of the construction: on a match it returns payload , on a non-match $0 \cdot \text{garbage}$.

The membership bit never appears in the clear to either party, so the join pattern is hidden from S. The masking multiplication takes the protocol out of the base paper’s “additive-only” regime by exactly one multiplicative level (plus relinearization), which must be accounted for in the modulus budget.

Remark 1 (circuit privacy). Since S’s computation touches the ciphertext before R decrypts the final analysis output, noise-flooding / circuit privacy must be applied before that decryption, exactly as the base protocol does on its returned ciphertexts.

3.3 Scheme split: indicator in BGV/BFV, payload in CKKS

Real-world payloads are typically real numbers, so the payload OKVS is naturally carried in CKKS while the indicator/equality logic stays in BGV/BFV. The re-encryption of $b_{X,i}$ into CKKS is free: the share is plaintext to its holder, so no scheme-switching primitive is required — R simply CKKS-encrypts a fresh 0/1, and (1) runs in CKKS. The only semantic change from the BGV masking is that *exact* zeroing becomes *approximate*: with $b_i = 0 \pm \delta$, a non-match leaves a residual $\delta \cdot \text{garbage}$. Hence the central numerical requirement:

$$\|p\| \leq \frac{\|v\|}{\sigma_{\min}(M)}, \quad |\text{garbage}| = |\langle \text{row}(y'), p \rangle| \leq \sqrt{w} \|p\|, \quad \text{CKKS residual} \sim \delta \cdot |\text{garbage}|,$$

so controlling $\sigma_{\min}(M)$ (equivalently $\|p\|$) is what keeps the residual below the analysis tolerance (Section 4).

3.4 Slot geometry and the “promote later” parameter plan

BGV/BFV with ring dimension N has N slots; CKKS with ring dimension N' has $N'/2$ slots. To share one sequencing grid between the two decodes (so R packs its membership share once, with no cross-grid permutation), the slot counts must match. The clean way is to choose the CKKS ring as $N' = 2N$, which CKKS likely needs anyway for the masking depth and payload precision. Then both have N slots and the (μ_i, τ_i) layout is identical (it depends only on H_1 , the slot count, and ζ ; w and the band values may differ between the two instances).

Two further points drive the parameterization:

- **Keep N small during alignment.** Sequencing is n -balls-into-(slot count); large slot counts under-fill the bins and waste computation on near-empty ciphertexts. The alignment phase should run at modest N (e.g. BFV 2^{13} / CKKS 2^{14}), preserving $N \ll n$.
- **Bootstrap is not a dimension lift.** N is fixed at key generation; bootstrapping refreshes the modulus/level at the *same* N . If deep analysis is needed, one keeps alignment at small N and, *after* masking (alignment done), ring-switches the aligned payload *up* to a large ring and bootstraps there (a sparse-slot, hence favorable, bootstrapping). The big- N bootstrapping keys must be provisioned upfront, but the shallow alignment work stays cheap at small N . Ring switching *down* (large→small) is the genuinely useful direction for shrinking the tail; ring switching *up* grants no fresh noise budget by itself — only the subsequent bootstrap does.

4 Real-field OKVS: conditioning instead of rank

For the payload OKVS over $\mathbb{R}$ (CKKS) we propose *binary* random bands: 0/1 coefficients keep the homomorphic decode addition-only and keep the garbage a bounded subset sum. The encoding

solves $Mp = v$ over $\mathbb{R}$, where M is $n \times m$, $m \approx (1 + \varepsilon)n$, each row a width- w binary band at a random start. Two facts reshape the analysis relative to the finite-field case.

Rank transfers for free; conditioning is the new question. A 0/1 matrix that is full row rank over $\text{GF}(2)$ has an $n \times n$ submatrix of odd determinant, which is therefore nonzero over $\mathbb{R}$; hence $\Pr_{\mathbb{R}}[\text{full rank}] \geq \Pr_{\text{GF}(2)}[\text{full rank}]$, and the (ε, w) from the finite-field analysis are a conservative choice for the rank event. But CKKS needs more than nonzero determinant: a full-rank M with tiny $\sigma_{\min}(M)$ still yields an exploding $\|p\|$. The operative target is therefore a *smallest-singular-value tail bound*

$$\Pr[\sigma_{\min}(M) < t] \leq 2^{-\lambda},$$

which replaces the rank-failure bound and which sets (ε, w) . Redundancy ε (extra columns) is the more natural knob than w for raising $\sigma_{\min}$.

Efficient minimum-norm encoding. Among all valid encodings $\{p_0 + \ker M\}$ the minimum- ℓ_2 -norm solution is $p^* = M^\top(MM^\top)^{-1}v$, and its size is governed precisely by $\sigma_{\min}$: $\|p^*\| \leq \|v\|/\sigma_{\min}(M)$. Crucially this is cheap to compute for band matrices: ordering rows by band start makes $MM^\top$ *banded* (bandwidth $O(w)$), so $MM^\top y = v$ is solved by banded Cholesky in $O(nw^2)$ and $p^* = M^\top y$ in $O(nw)$ — comparable to the encoding itself, and far smaller in norm than the naive triangular particular solution (free variables set to 0), whose norm is governed by the worse-conditioned pivot submatrix. (Normal equations square the condition number; a banded QR of $M^\top$ is the stable fallback.)

5 Open problems / next steps

Open problem 1 ($\sigma_{\min}$ tail bound). Prove $\Pr[\sigma_{\min}(M) < t] \leq 2^{-\lambda}$ for random width- w binary band matrices with $m = (1 + \varepsilon)n$, giving the (ε, w, t) trade-off. (Toolbox: sparse/structured $\sigma_{\min}$ bounds à la Rudelson–Vershynin, Litvak–Pajor–Rudelson–Tomczak-Jaegermann.)

Open problem 2 (empirical conditioning). Measure, for binary band matrices over $\mathbb{R}$: the amplification $\|p\|/\|v\|$ for naive-triangular vs. banded-min-norm; $\sigma_{\min}(M)$ and its dependence on (w, ε) ; and the resulting garbage magnitude $|\langle \text{row}(y'), p \rangle|$ on non-matches. (Addressed by the accompanying experiment.)

Open problem 3 (2-row decode). The $2 \times (N/2)$ BGV hypercube admits, with $N/2$ -based sequencing and rotation-during- compute, two layer products per row-wise rotation — halving plain-mults for the indicator decode and resolving the open remark of §4.3 of the base paper. The packing (offset alignment of paired layers; replication vs. windowed layout) and its communication/computation trade-off remain to be fixed.